```
POST --data-urlencode \'payload=${payload}\' ${slack_url}"
}

pipeline {
     agent {
       node {
           customWorkspace "$jenkins_node_custom_workspace_
path"
           label "$jenkins_node_label"
       }
     }

     stages {
       stage('fetch_latest_code') {
           steps {
               git branch: "$git_branch" ,
               credentialsId: "$credentials_id" ,
               url: "$git_url"
           }
       }

       stage('install_deps') {
           steps {
               sh "sudo apt install wget zip python-pip -y"
               sh "cd /tmp"
               sh "curl -o terraform.zip https://
releases.hashicorp.com/terraform/'$terraform_version'/
terraform_'$terraform_version'_linux_amd64.zip"
               sh "unzip terraform.zip"
               sh "sudo mv terraform /usr/bin"
               sh "rm -rf terraform.zip"
           }
       }

       stage('init_and_plan') {
           steps {
               sh "sudo terraform init $jenkins_node_custom_
workspace_path/workspace"
               sh "sudo terraform plan $jenkins_node_custom_
workspace_path/workspace"
               notifySlack("Build completed! Build logs
from jenkins server $jenkins_server_url/jenkins/job/$JOB_
NAME/$BUILD_NUMBER/console", notification_channel, [])
           }
       }

       stage('approve') {
           steps {
             notifySlack("Do you approve deployment?
$jenkins_server_url/jenkins/job/$JOB_NAME", notification_
channel, [])
               input 'Do you approve deployment?'
           }
       }

       stage('apply_changes') {
           steps {
               sh "echo 'yes' | sudo terraform apply
$jenkins_node_custom_workspace_path/workspace"
               notifySlack("Deployment logs from jenkins
server $jenkins_server_url/jenkins/job/$JOB_NAME/$BUILD_
NUMBER/console", notification_channel, [])
           }
       }
     }

     post {
     always {
     cleanWs()
   }
  }
}
```

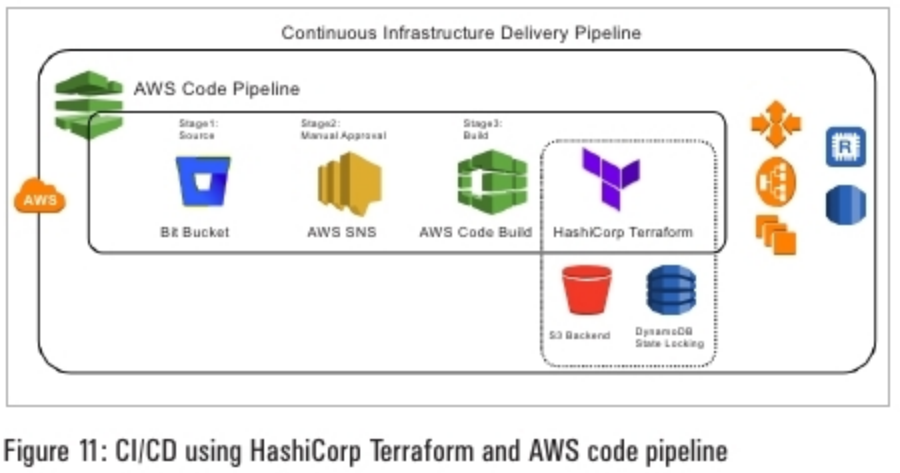


Figure 11: CI/CD using HashiCorp Terraform and AWS code pipeline

It is recommended that you use reusable modules in Terraform by writing your own modules, using modules from the Terraform registry. We can also use the Docker build agent for the Jenkins slave and save the workspace by attaching a persistent volume to the Jenkins server from the Docker host. It is recommended that readers encrypt the Consul key-value with HashiCorp Vault. This is a reliable key management service and can be accessed by *http* calls.

Right now, all cloud providers are offering their own CI tools. AWS offers CodePipeLine, with which we can use Code Commit for *scm*, Code Build for the build environment (where we can apply Terraform configurations), and SNS to send notifications for manual approval. Azure offers Azure DevOps tools for creating the CI/CD pipeline, whereby the user can commit the code to Azure TFS or any *scm* through *vsts* and trigger the CI job. We can set up the pipeline job based on the cloud platform being used. Here Jenkins can be used in the cloud as well as in the on-premises infrastructure. END

**By: Radhakrishnan Ramakrishnan**

The author is a DevOps engineer with Dattatreya Consultancy Services Pvt Ltd, Madurai. He has expertise in Linux, OpenStack, AWS, Web development and Hadoop administration.